NEXUS

Plateforme de collaboration interentreprises

Année universitaire 2026–2027

MEMBRES DE GROUPES



Latrech Arij



Ben Boubaker Sarra



Ben Hamouda Yoser



Dawahi Med Aziz



Mazitti Chaima



Mihoubi Akrem

PLAN DE LA PRÉSENTATION

01. Présentation du projet

02. Acteurs et fonctionnalités

03. Architecture globale

04. Découpage en microservices

05. Technologies utilisées

06. Organisation de l'équipe

07. Planning et gestion des risques

08. Présentation des microservices

01. PRÉSENTATION DU PROJET :

Nexus permet à plusieurs entreprises de collaborer sur des projets communs depuis une seule plateforme : gérer le projet, répartir les tâches, partager des documents et leurs versions, échanger des commentaires, faire valider les livrables et garder un historique fiable des actions importantes.

1.1 PROBLÈME:

La collaboration entre plusieurs entreprises devient difficile lorsque les tâches, les documents, les discussions et les validations sont dispersés dans différents outils.

1.2 .OBJECTIF :

centraliser la collaboration, faciliter le suivi des projets et sécuriser les échanges entre entreprises partenaires.

02 .ACTEURS ET FONCTIONNALITÉS

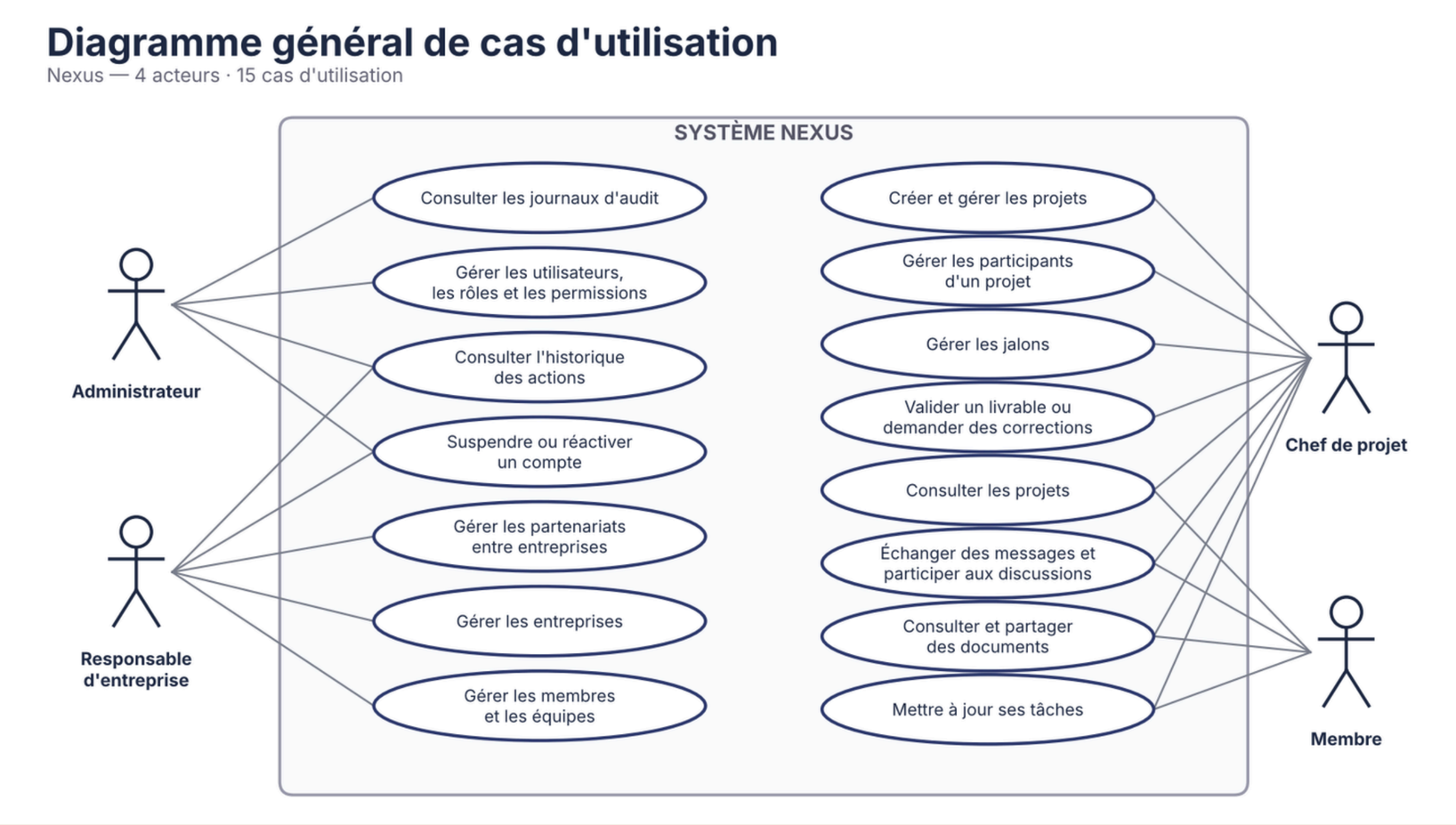
2.1. Acteurs du système :

ACTEUR	RESPONSABILITÉ
Administrateur	Gestion globale de la plateforme.
Responsable d'entreprise	Gestion de l'organisation, des membres et des équipes.
Chef de projet	Coordination des projets et suivi des tâches.
Membre	Participation aux projets et réalisation des tâches.

2.2. Les fonctionnalités principales.

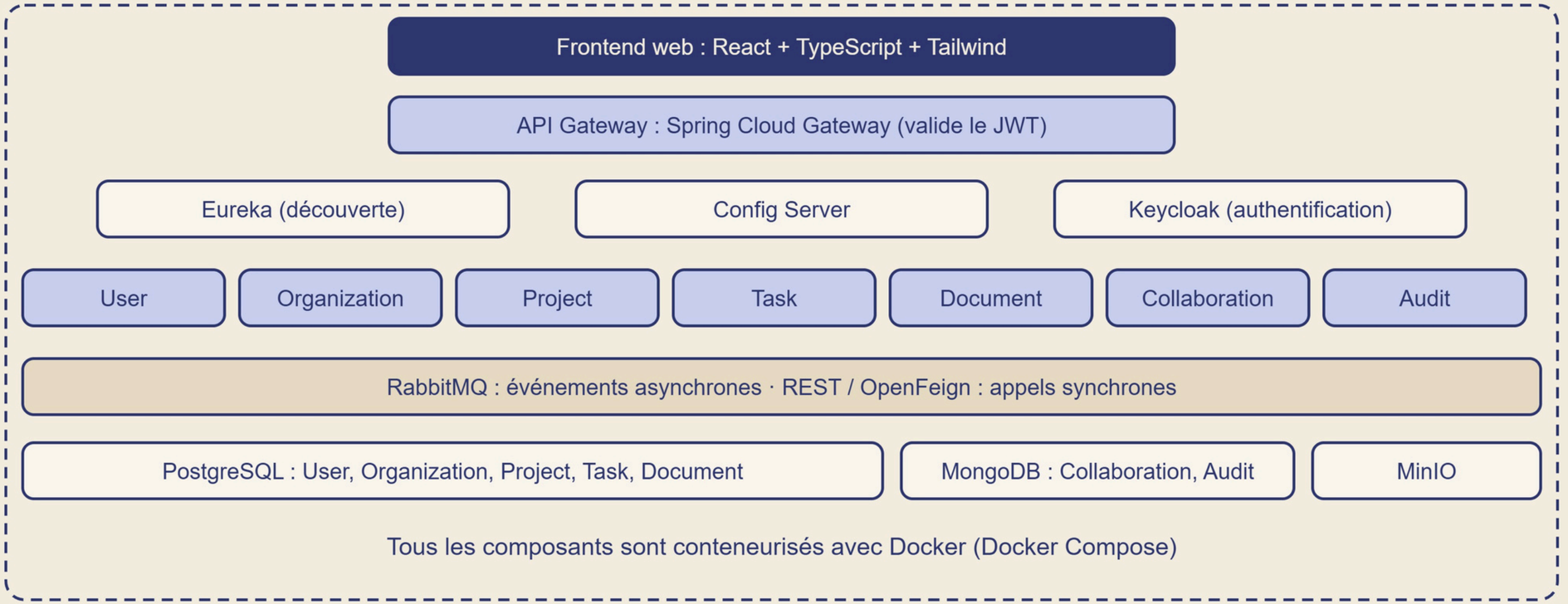
MICROSERVICE	FONCTIONNALITÉS
Organization	Gestion des entreprises, des membres, des équipes et des partenariats.
Project	Création des projets, gestion des participants et suivi des jalons.
Task	Création, attribution et suivi des tâches.
Document	Dépôt, partage et gestion des documents et des autorisations.
Collaboration	Communication et échanges entre les membres des projets.
Audit	Traçabilité des actions et historique des modifications.
User	Gestion des comptes, authentification, rôles et permissions.

2.3. Diagramme général de cas d'utilisation.



03. ARCHITECTURE GLOBALE

La plateforme utilise React et une API Gateway sécurisée par JWT (Keycloak). Eureka assure la découverte des services et Config Server centralise leurs configurations. Chaque microservice dispose de sa propre base de données (PostgreSQL ou MongoDB). Les échanges se font via REST/OpenFeign ou RabbitMQ. MinIO gère les fichiers et Docker Compose assure la conteneurisation.



04. DÉCOUPAGE EN MICROSERVICES

SERVICE	RÔLE	EXEMPLE CONCRET	ÉVÉNEMENTS (EXEMPLES)
1. User Service	Comptes, connexion, jeton court + refresh, rôles globaux (Super Admin, Admin d'entreprise, Utilisateur). Ne gère pas les rôles de projet	Inscrire un utilisateur et lui attribuer un rôle global.	AccountCreated AccountSuspended
2. Organization Service	Gère les entreprises, les équipes et leurs membres.	Ajouter une équipe dans une entreprise.	OrganizationCreated OrganizationUpdated
3. Project Service	Gère les projets, leurs informations et leurs participants.	Créer un projet et inviter des membres.	ProjectCreated ProjectUpdated
4. Task Service	Gère les tâches, leur attribution, leurs priorités et leurs échéances.	Attribuer une tâche à un développeur.	TaskCreated TaskCompleted

SERVICE	RÔLE	EXEMPLE CONCRET	ÉVÉNEMENTS (EXEMPLES)
5. Document Service	Gère les fichiers, les contrats et leurs versions.	Déposer la version 2 d'un contrat.	DocumentUploaded DocumentVersionAdded
6. Collaboration Service	Gère les discussions, les commentaires et les mentions.	Mentionner un collègue dans un commentaire.	CommentAdded MentionCreated
7. Audit Service	Conserve l'historique des actions importantes.	Tracer une connexion ou une modification.	AuditLogged SecurityEvent

05 .TECHNOLOGIES UTILISÉES

5.1. Backend

Java 17 / Spring Boot

Développement des microservices et implémentation de la logique métier.

Spring Security + JWT

Authentification, gestion des rôles et sécurisation des API.

Spring Data JPA / Hibernate

Gestion des entités et interactions avec les bases de données.

Keycloak

Gestion de l'authentification et des autorisations

Config Server

Configuration centralisée

Eureka

Découverte des microservices

5.2. Frontend

React.js
Création d'une interface utilisateur dynamique et interactive.



JavaScript / TypeScript
Développement de la logique côté client et structuration du code.



Tailwind CSS
Création d'une interface moderne, responsive et cohérente.



5.3. Bases de données et stockage



PostgreSQL
Stockage des données relationnelles de chaque microservice (utilisateurs, projets, tâches, documents, etc.).



MinIO
Stockage des fichiers et des documents partagés, avec gestion de leurs différentes versions.



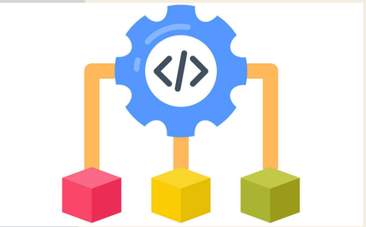
MongoDB
Stockage des données non relationnelles et semi-structurées, notamment les messages, les discussions et les notifications échangés entre les utilisateurs

5.4. Communication entre microservices

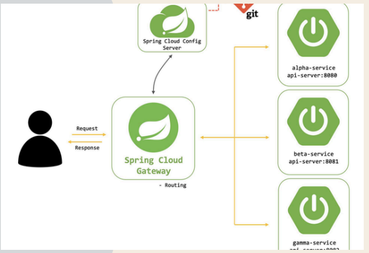
RabbitMQ – Communication asynchrone
Échange d'événements entre microservices sans attendre une réponse immédiate.



REST API – Communication synchrone
Échange direct de données entre services à travers des requêtes HTTP.



Spring Cloud Gateway
Point d'entrée unique qui centralise le routage des requêtes vers les microservices.



5.5 . Outils de développement et déploiement



GitHub



Docker



Postman



IntelliJ IDEA



Swagger



VS Code

5.6 . Dépôt Git et organisation du projet

Gestion du code source avec GitHub

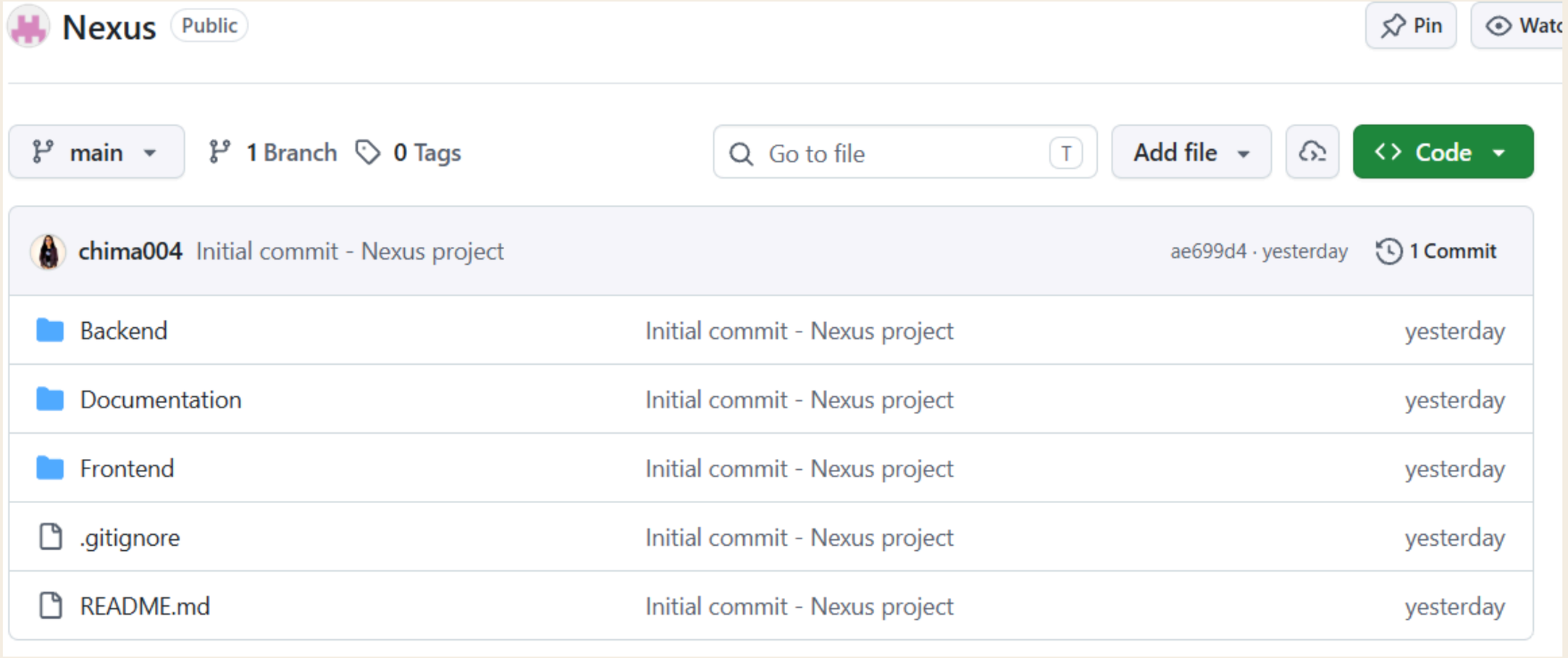
Le projet NEXUS est organisé dans un dépôt GitHub public afin de faciliter le travail collaboratif, le suivi des modifications et la répartition des tâches entre les membres de l'équipe.

Structure du dépôt

Structure

```
nexus_4DAS16-g4/
├── Backend/
│   ├── organisation/
│   ├── projet/
│   ├── task/
│   ├── document/
│   ├── collaboration/
│   ├── user/
│   └── audit/
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   ├── Diagrams/
│   └── Dossier-technique/
└── README.md
```

5.6 . Dépôt Git et organisation du projet



06. ORGANISATION DE L'ÉQUIPE

Répartition des responsabilités

MEMBRE	MICROSERVICE
Latrech Arij	Organization
Dawahi Med Aziz	Project
Maziti Chaima	Task
Boubaker Sarra	Collaboration
Mihoubi Akrem	Document
Ben Hamouda Yosr	Audit
Toute l'équipe	User

07. PLANNING ET GESTION DES RISQUES

7.1 Étapes de développement.

PHASE 1 — Analyse des besoins
Identification des besoins fonctionnels, des acteurs et des exigences du système.

PHASE 2 — Conception et architecture
Modélisation UML, définition de l'architecture microservices et conception des bases de données.

PHASE 3 — Développement Backend
Implémentation des 7 microservices, des API REST et des mécanismes de sécurité.

PHASE 4 — Développement Frontend
Conception de l'interface utilisateur et intégration avec les services Backend.

PHASE 5 — Intégration des services
Configuration de l'API Gateway et mise en place de la communication entre les microservices.

PHASE 6 — Tests et validation
Réalisation des tests unitaires, d'intégration et fonctionnels afin de vérifier la fiabilité du système.

PHASE 7 — Déploiement
Conteneurisation avec Docker, configuration de l'environnement et mise en service de la plateforme.

7.2. Difficultés potentielles et solutions

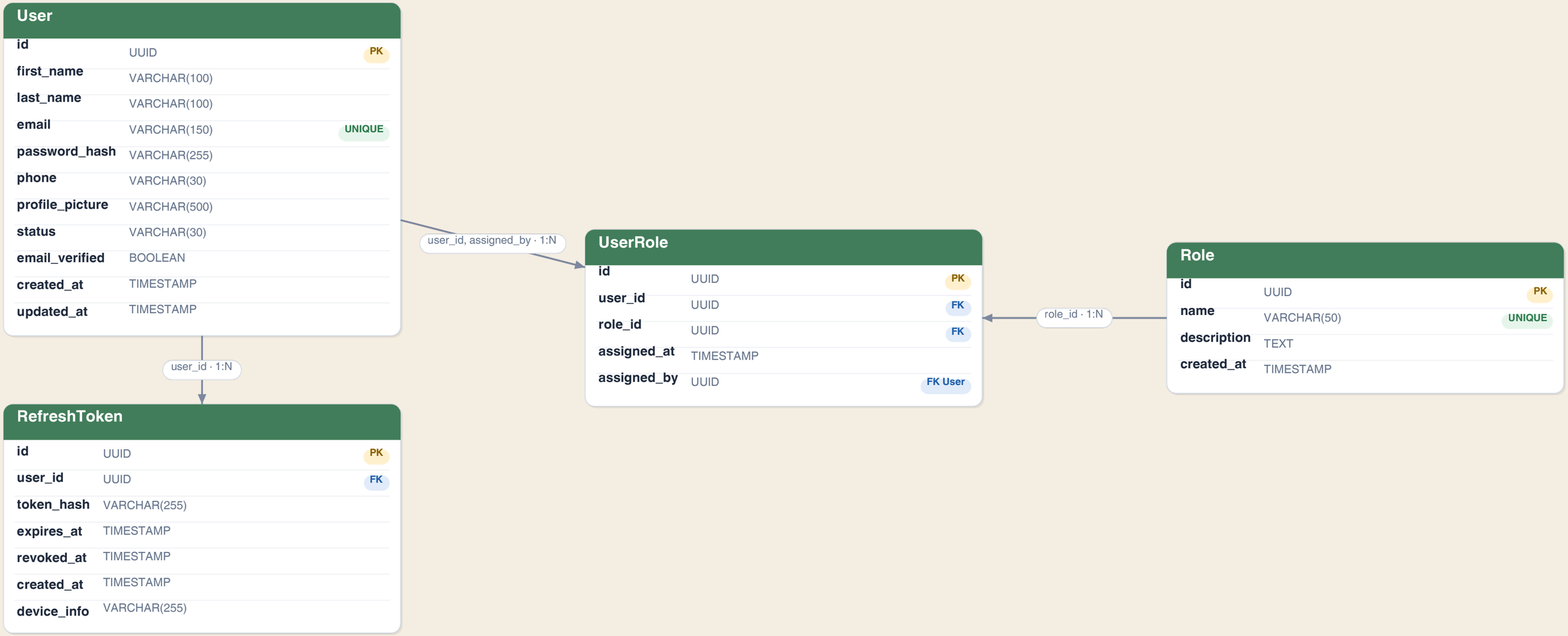
DÉFI	SOLUTION
Coordination des 7 microservices	Contrats d'API documentés avec Swagger/OpenAPI, conventions communes (noms d'événements, format des erreurs), workflow GitHub (branches + pull requests), réunions de synchronisation régulières
Cohérence des données entre bases indépendantes	Une base par service, références par identifiant (UUID) au lieu de jointures, événements RabbitMQ pour synchroniser, cohérence éventuelle, consommateurs idempotents
Gestion des droits d'accès entre entreprises	Rôles globaux (User Service) + rôles de projet (Project Service), JWT contenant userId, entreprise et rôle, vérification à l'API Gateway puis dans chaque service (@PreAuthorize), contrôle du partenariat via Organization Service
Sécurisation du partage des documents	Vérification des autorisations avant tout accès, URLs pré-signées MinIO à durée limitée, HTTPS, contrôle du type et de la taille des fichiers, versionnage des documents

7.2. Difficultés potentielles et solutions

DÉFI	SOLUTION
Échanges synchrones et asynchrones	Règle claire : REST quand une réponse immédiate est nécessaire, RabbitMQ sinon. Timeouts, retry et circuit breaker (Resilience4j), dead-letter queues pour les messages en échec
Centralisation et traçabilité des actions	Audit Service qui consomme tous les événements, format d'événement standard (qui, quoi, quand, ressource), journal en ajout seul (non modifiable), identifiant de corrélation entre services
Intégration et déploiement de l'architecture distribuée	Conteneurisation Docker de chaque service, Docker Compose pour l'environnement complet, configuration par variables d'environnement, tests d'intégration, pipeline CI avec GitHub Actions

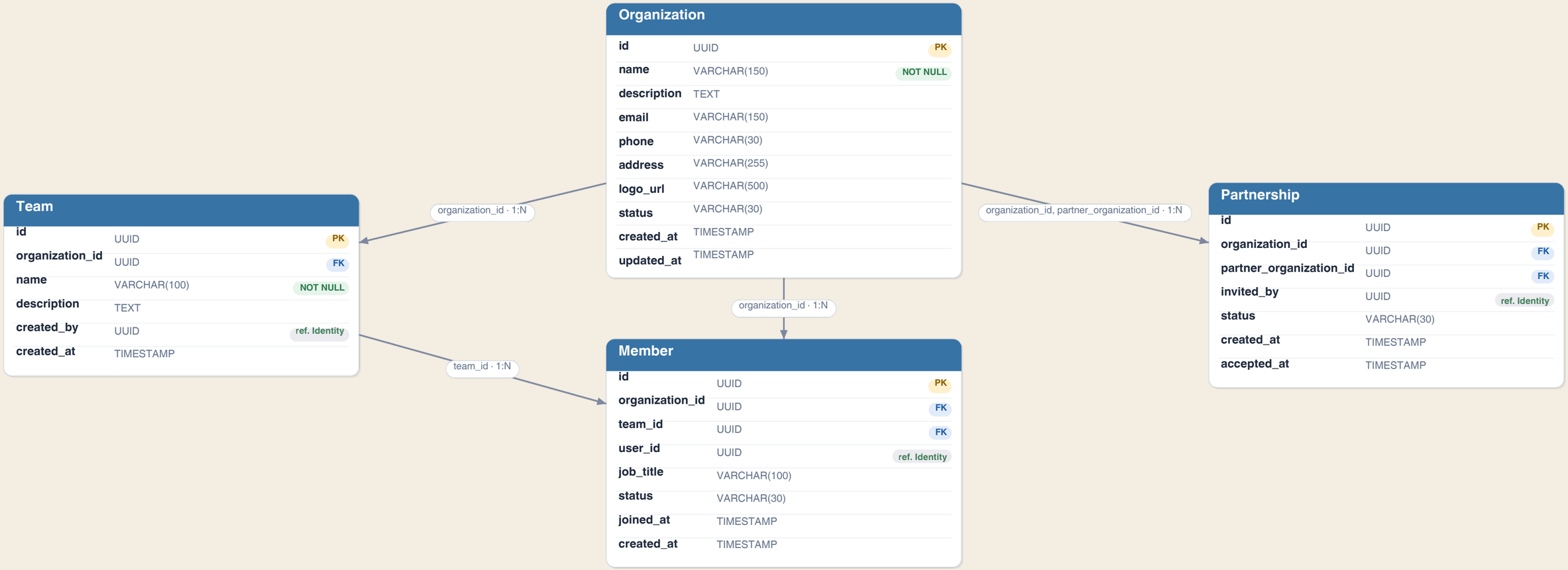
08. PRÉSENTATION DES MICROSERVICES

USER SERVICE



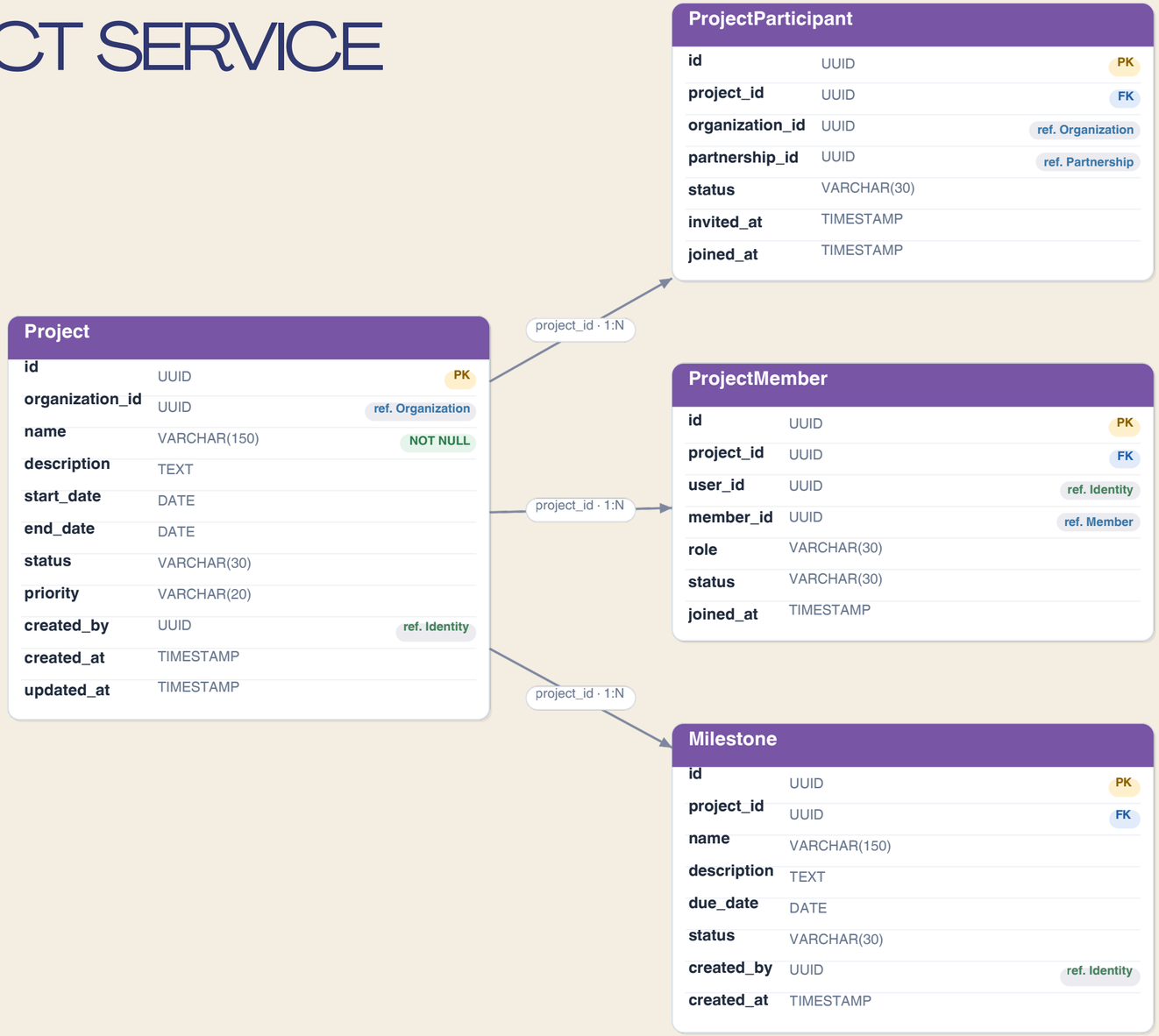
08. PRÉSENTATION DES MICROSERVICES

ORGANIZATION SERVICE



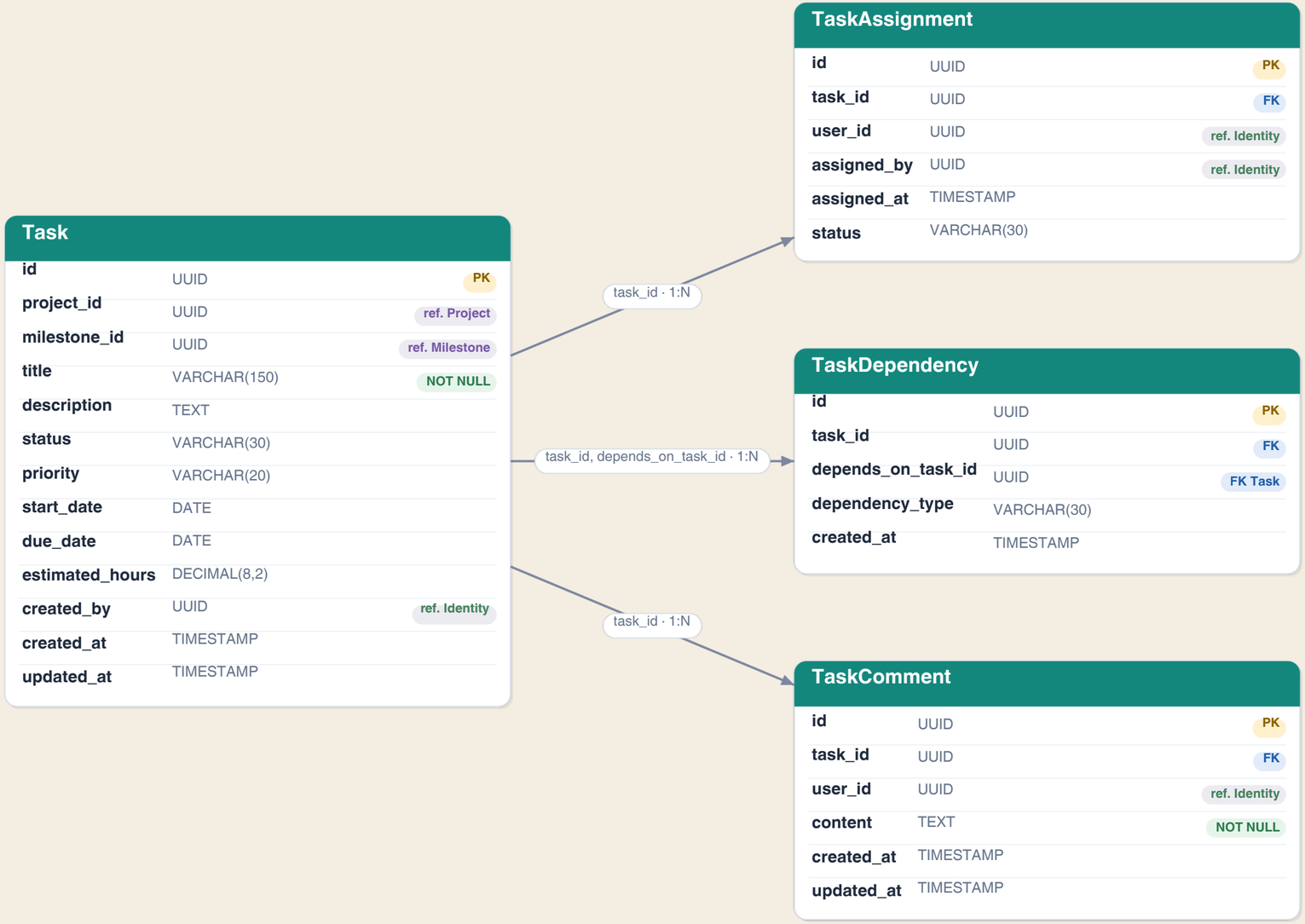
08. PRÉSENTATION DES MICROSERVICES

PROJECT SERVICE



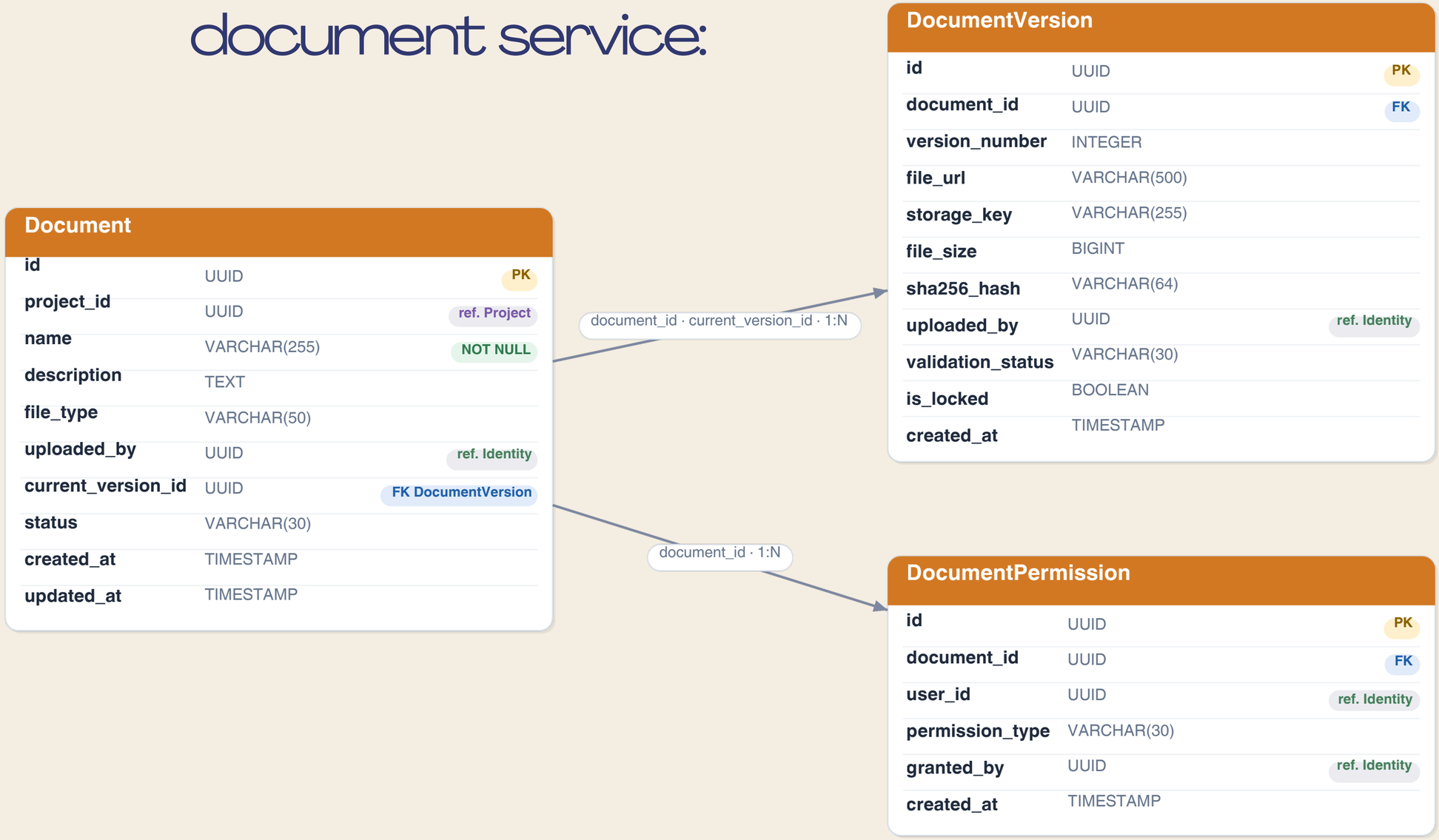
08. PRÉSENTATION DES MICROSERVICES

TASK SERVICE



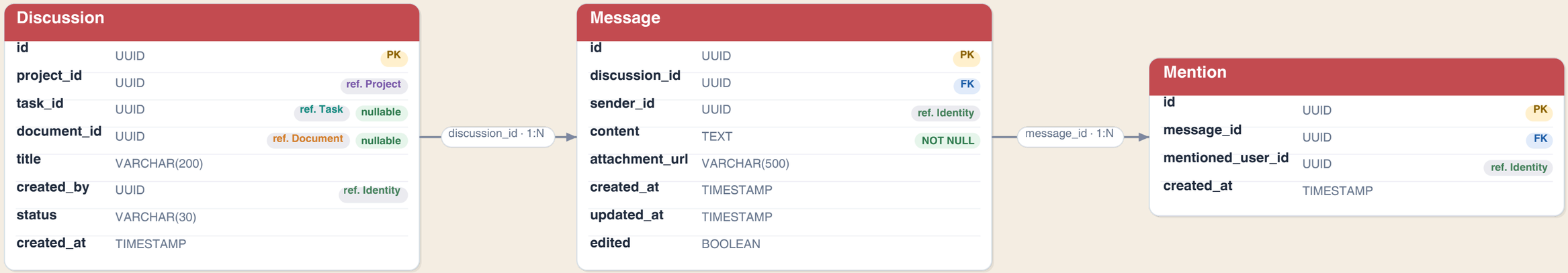
08. PRÉSENTATION DES MICROSERVICES

document service:



08. PRÉSENTATION DES MICROSERVICES

COLLABORATION SERVICE



08. PRÉSENTATION DES MICROSERVICES

audit service:

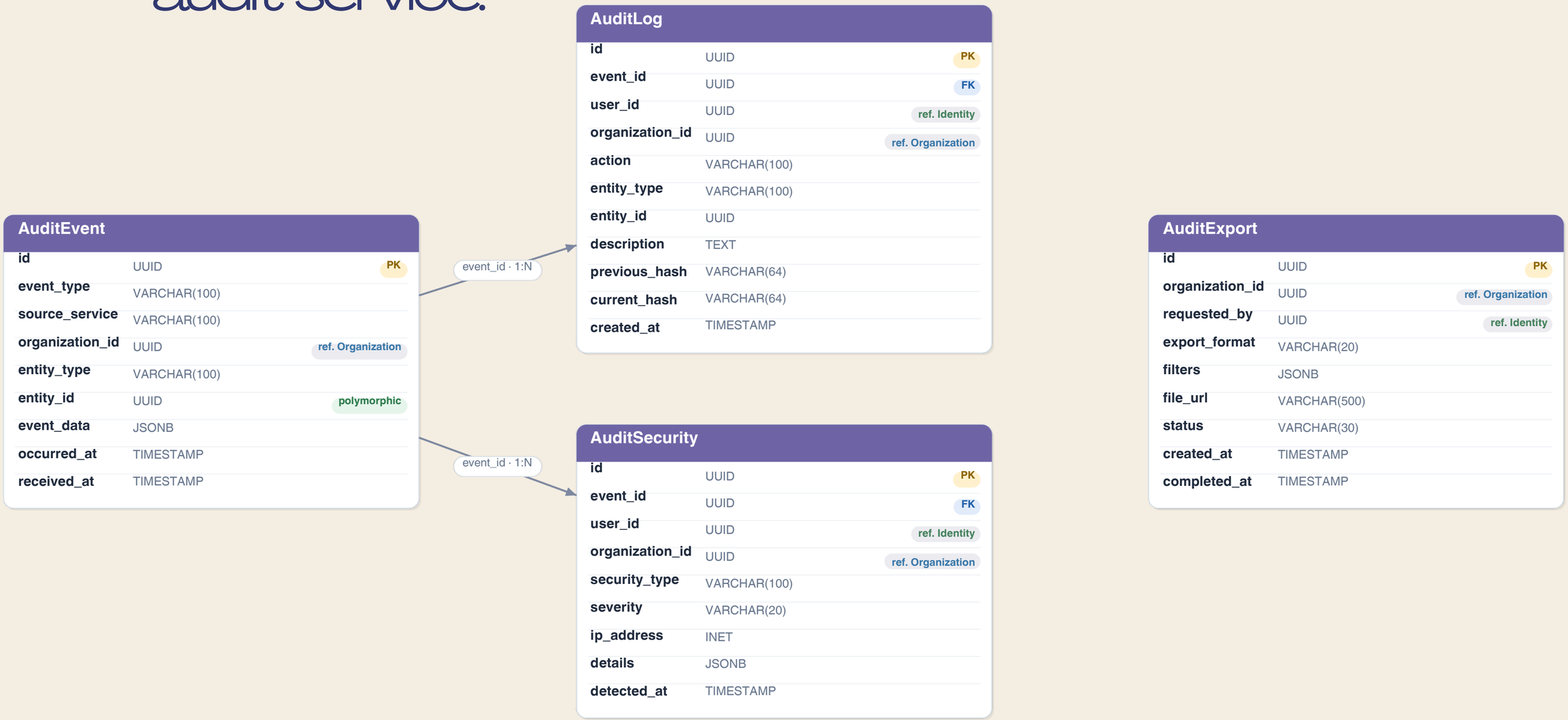
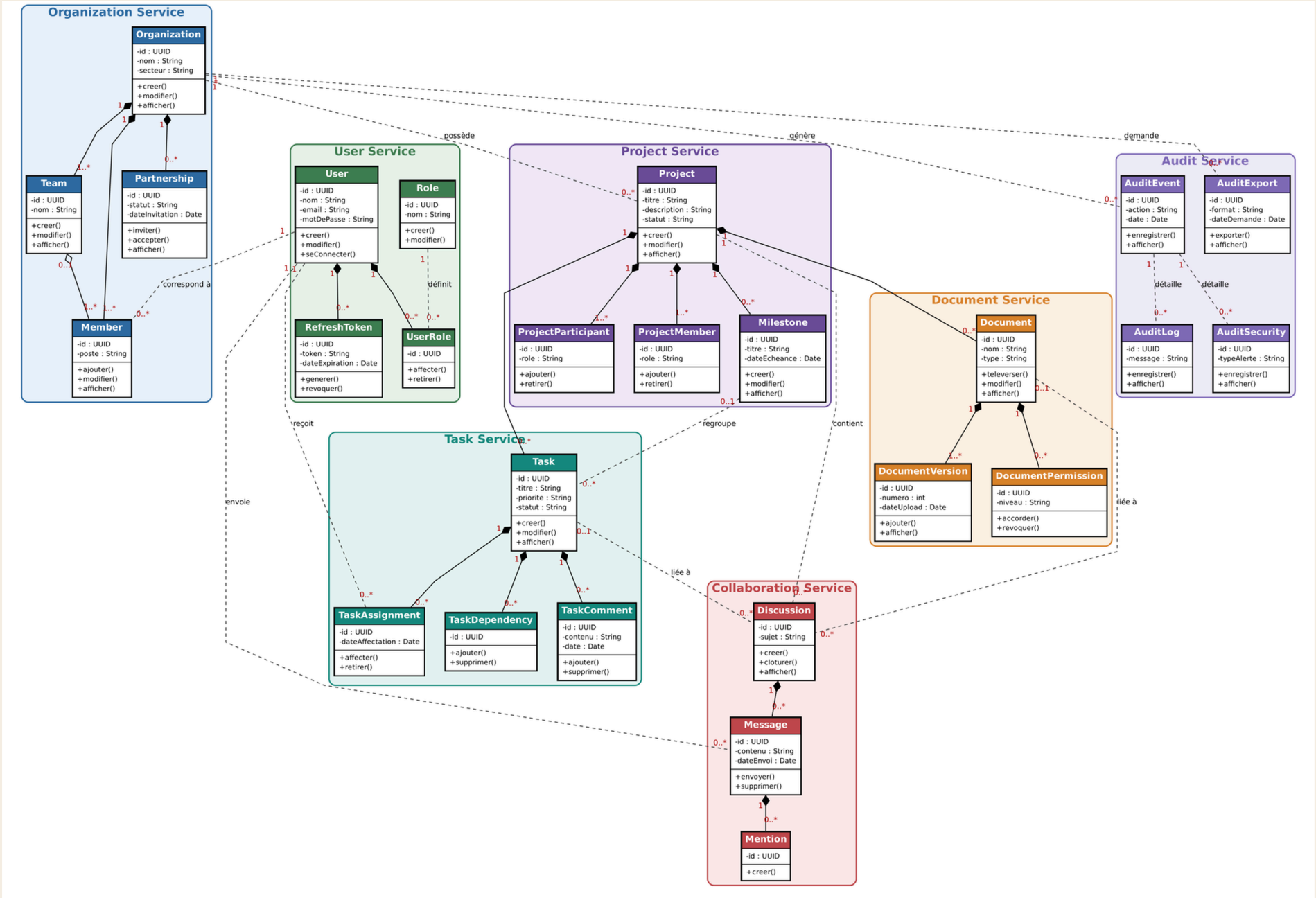


DIAGRAMME DE CLASSES GLOBALE



08. PRÉSENTATION DES MICROSERVICES

API REST principales

MICROSERVICE	MÉTHODE	ENDPOINT	FONCTION
User	POST	/api/users	Créer un utilisateur
Organization	POST	/api/organizations	Créer une entreprise
Project	POST	/api/projects	Créer un projet
Task	POST	/api/tasks	Créer une tâche
Document	POST	/api/documents	Déposer un document
Collaboration	POST	/api/messages	Envoyer un message
Audit	GET	/api/audit/logs	Consulter l'historique

08. PRÉSENTATION DES MICROSERVICES

Interactions entre microservices

SERVICE SOURCE	SERVICE DESTINATAIRE	COMMUNICATION	EXEMPLE
Organization	User	REST / OpenFeign	Vérifier l'existence d'un utilisateur avant de l'ajouter comme membre.
Project	Organization	REST / OpenFeign	Vérifier l'existence d'une entreprise partenaire.
Task	Project	REST / OpenFeign	Vérifier l'existence du projet associé à une tâche.
Document	Project	REST / OpenFeign	Vérifier l'existence du projet auquel appartient le document.
Collaboration	Project	REST / OpenFeign	Vérifier l'existence du projet associé à une discussion.
Collaboration	Task	REST / OpenFeign	Vérifier l'existence d'une tâche associée à une discussion.

08. PRÉSENTATION DES MICROSERVICES

Interactions entre microservices

SERVICE SOURCE	SERVICE DESTINATAIRE	COMMUNICATION	EXEMPLE
Collaboration	Document	REST / OpenFeign	Vérifier l'existence d'un document associé à une discussion.
User	Audit	RabbitMQ	Enregistrer une connexion ou une modification de compte.
Organization	Audit	RabbitMQ	Enregistrer la création ou la modification d'une entreprise.
Project	Audit	RabbitMQ	Enregistrer la création ou la modification d'un projet.
Task	Audit	RabbitMQ	Enregistrer la création ou la finalisation d'une tâche.
Document	Audit	RabbitMQ	Enregistrer le dépôt ou la modification d'un document.
Collaboration	Audit	RabbitMQ	Enregistrer l'envoi d'un message ou la création d'une mention.

CONCLUSION

NEXUS est une plateforme centralisée facilitant la collaboration entre entreprises à travers la gestion des projets, tâches, documents et échanges. Son architecture microservices garantit modularité, sécurité, évolutivité et maintenabilité.